

UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA,
ELÉCTRICA, ELECTRÓNICA Y SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



INFORME TÉCNICO: PRÁCTICA FINAL DE UNIDAD I

Caso Integrador: Mini Hoja de Cálculo Académica de Alta Performance

CURSO

Algoritmos y Estructuras de Datos

AUTOR

Apaza Calizaya Rodrigo Josseph

Nombre del docente
ZANABRIA GALVEZ ALDO HERNAN

Puno, Perú
2026

Índice

1. Introducción	2
2. Marco Teórico	2
3. Arquitectura del Sistema Integrado	2
4. Actividad 1: Mapeo Arquitectónico del Caso	3
5. Actividad 2: Cuantificación Compleja Asintótica	3
6. Actividad 3: Cuestionario Teórico Automatizado	3
6.1. Cuestión 1: Limitaciones de Listas Simples o Arreglos Unidimensionales . .	3
6.2. Cuestión 2: Degradación Asintótica de Tablas Hash	4
6.3. Cuestión 3: Optimización del Sistema Deshacer (Stack)	4
6.4. Cuestión 4: Gestión de Memoria en la Operación Deshacer (C++ vs Python)	4
7. Comparativa entre C++ y Python	4
8. Conclusiones	5
9. Referencias	5
10. Anexos	5

1. Introducción

El presente informe técnico documenta el diseño conceptual e implementación física de una **Mini Hoja de Cálculo Académica** orientada a consola de comandos. Las hojas de cálculo modernas operan bajo restricciones críticas de rendimiento e integridad, requiriendo el uso sincronizado de múltiples representaciones abstractas de datos en memoria volátil.

Para resolver este desafío integrador, se desarrolló un software robusto en C++ fundamentado en estructuras híbridas de acceso dinámico ($O(1)$ y $O(\log n)$), combinando vectores secuenciales, indexación asociativa determinista mediante tablas hash, resguardos LIFO en pilas para el control transaccional del sistema de deshecho, y flujos asíncronos FIFO encapsulados en colas de atención para la simulación de procesos.

2. Marco Teórico

Las hojas de cálculo modernas constituyen sistemas de procesamiento de información que requieren estructuras de datos eficientes para almacenar, localizar y transformar grandes volúmenes de registros en memoria. La eficiencia de estos sistemas depende directamente de la selección adecuada de las estructuras subyacentes y de los algoritmos responsables del acceso y manipulación de los datos.

Las listas dinámicas permiten almacenar registros de manera secuencial ofreciendo acceso indexado en tiempo constante. Por su parte, las tablas hash proporcionan un mecanismo de búsqueda altamente eficiente mediante funciones de dispersión que asocian claves únicas con posiciones físicas en memoria.

Las pilas constituyen estructuras de tipo LIFO (Last In, First Out), ampliamente utilizadas para implementar sistemas de reversión de acciones. En contraste, las colas siguen una política FIFO (First In, First Out), resultando adecuadas para modelar procesos de atención secuencial.

Finalmente, los algoritmos de ordenamiento representan un componente esencial en el análisis de datos. QuickSort destaca por su excelente rendimiento promedio, mientras que MergeSort ofrece estabilidad y un comportamiento predecible incluso en escenarios adversos.

3. Arquitectura del Sistema Integrado

La solución desarrollada utiliza una arquitectura híbrida donde múltiples estructuras de datos colaboran de manera coordinada.

La lista dinámica actúa como almacenamiento principal de registros estudiantiles. Sobre ella se construye una tabla hash que mantiene una relación entre el código único del estudiante y su posición física dentro del vector. Esta sincronización permite conservar la eficiencia tanto en recorridos secuenciales como en búsquedas directas.

Las operaciones de modificación generan instantáneas almacenadas dentro de una pila, permitiendo la implementación de un mecanismo de reversión de cambios. De manera

complementaria, la cola de atención académica administra solicitudes pendientes siguiendo una política FIFO.

La integración de estas estructuras reproduce principios similares a los empleados en sistemas empresariales y hojas de cálculo modernas.

4. Actividad 1: Mapeo Arquitectónico del Caso

Componente Académico	Estructura Base	Justificación Arquitectónica
Persistencia de Celdas	Vector Dinámico	Mapeo contiguo en memoria ram para lecturas tabulares.
Búsqueda de Código	Tabla Hash	Indexación en tiempo constante evitando barridos costosos.

5. Actividad 2: Cuantificación Compleja Asintótica

Operación	Estructura	Tiempo	Espacio
-----------	------------	--------	---------

Registrar estudiante	Lista + Hash	$O(n)$	$O(1)$
Mostrar hoja completa	Lista dinámica	$O(n)$	$O(1)$
Buscar estudiante	Tabla Hash	$O(n)$ peor caso	$O(1)$
Ordenar rendimiento	QuickSort	$O(n^2)$	$O(n)$
Ordenar código	MergeSort	$O(n \log n)$	$O(n)$
Deshacer cambios	Pila	$O(1)$	$O(n)$
Encolar alumno	Cola	$O(1)$	$O(1)$
Despachar alumno	Cola	$O(1)$	$O(1)$
Promedio acumulado	Recursividad	$O(n)$	$O(n)$

6. Actividad 3: Cuestionario Teórico Automatizado

6.1. Cuestión 1: Limitaciones de Listas Simples o Arreglos Unidimensionales

Respuesta: Las dimensiones de las celdas en hojas de cálculo comerciales requieren mapeos directos y acceso bidimensional eficiente. El uso exclusivo de una lista enlazada simple destruye la *localidad de referencia espacial* en la memoria caché del procesador y descarta el acceso aleatorio directo indexado, obligando a búsquedas con costo lineal $O(n)$ mediante un costoso recorrido de punteros dispersos en memoria dinámica. Un arreglo unidimensional básico, por otro lado, requeriría costosas operaciones de traslación manual de bloques completos de bytes ante inserciones o eliminaciones intermedias de filas.

6.2. Cuestión 2: Degradación Asintótica de Tablas Hash

Respuesta: El factor de carga está determinado formalmente por la relación $\alpha = \frac{n}{m}$. Cuando ocurre una colisión sistemática severa provocada por una función de dispersión deficiente, múltiples claves se asignan al mismo contenedor (*bucket*). Bajo el método de resolución por encadenamiento lineal, el tiempo de búsqueda esperado degenera matemáticamente a:

$$T(n) = \mathcal{O}(1) + \mathcal{O}(\alpha) \quad (1)$$

En el peor escenario teórico, la lista interna del casillero colisionado contendrá los n elementos del sistema, obligando a una inspección lineal secuencial completa, lo que transmuta la velocidad óptima promedio de $\mathcal{O}(1)$ a una ineficiente complejidad de peor caso $\mathcal{O}(n)$.

6.3. Cuestión 3: Optimización del Sistema Deshacer (Stack)

Respuesta: Almacenar copias íntegras del estado del vector principal (*Snapshot Pattern*) penaliza drásticamente el espacio auxiliar elevándolo a un costo insostenible de $\mathcal{O}(n)$ por cada interacción o cambio menor. Para optimizar el subsistema, se propone implementar el **Patrón Memento de grano fino** o un registro cronológico de operaciones inversas (*Command Log*). Al registrar en el Stack únicamente las diferencias o deltas del cambio atómico (por ejemplo, el identificador de celda, la operación efectuada y el valor previo), el costo de almacenamiento espacial por acción disminuye drásticamente a un costo constante optimizado de $\mathcal{O}(1)$.

6.4. Cuestión 4: Gestión de Memoria en la Operación Deshacer (C++ vs Python)

Respuesta: En el lenguaje C++, la invocación de la directiva `historial.pop()` ejecuta de manera determinista e inmediata la liberación de los recursos gracias al paradigma **RAII** (*Resource Acquisition Is Initialization*); el vector descartado de la pila destruye sus elementos liberando de inmediato la memoria del montón (*Heap*). En contraste, en **Python**, la eliminación del nodo superior únicamente reduce en uno el contador interno de referencias del objeto. El espacio físico no se libera inmediatamente en el sistema operativo; queda supeditado a los ciclos heurísticos internos de ejecución asíncrona del colector de basura (**Garbage Collector**), volviendo la administración de recursos un proceso no determinista.

7. Comparativa entre C++ y Python

La implementación principal fue desarrollada en C++; sin embargo, el caso de estudio también puede ser implementado en Python utilizando estructuras equivalentes.

C++ ofrece mayor control sobre la memoria y un mejor rendimiento computacional gracias a su naturaleza compilada. Las estructuras `vector`, `unordered_map`, `stack` y `queue` proporcionan eficiencia y tipado estático, reduciendo errores durante la ejecución.

Por otro lado, Python simplifica considerablemente el desarrollo mediante listas, diccionarios y colecciones dinámicas incorporadas. Aunque presenta una velocidad inferior debido a su interpretación, mejora la productividad y la legibilidad del código.

Desde una perspectiva académica, C++ resulta apropiado para estudiar el comportamiento interno de las estructuras de datos, mientras que Python facilita el desarrollo rápido de prototipos y aplicaciones orientadas al análisis de información.

8. Conclusiones

1. La integración de listas dinámicas, tablas hash, pilas, colas y algoritmos de ordenamiento permitió construir una solución académica eficiente para la gestión de registros estudiantiles.
2. El análisis asintótico facilitó la evaluación objetiva del rendimiento de cada operación, permitiendo seleccionar las estructuras más adecuadas para cada requerimiento funcional.
3. QuickSort y MergeSort demostraron ventajas complementarias, destacando respectivamente por su velocidad promedio y estabilidad.
4. La comparación entre C++ y Python evidenció el equilibrio existente entre rendimiento computacional y facilidad de desarrollo.
5. Las estructuras de datos constituyen el núcleo operativo de sistemas complejos como hojas de cálculo, ERP académicos y plataformas empresariales modernas.

9. Referencias

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Morin, P. (2013). *Open Data Structures: An Introduction*. Athabasca University Press.
3. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.
4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
5. Python Software Foundation. (2026). *Data Structures*. Python Documentation.
6. Python Software Foundation. (2026). *collections — Container datatypes*. Python Documentation.

10. Anexos